

report.html, explained piece by piece

A walkthrough of every block of code in report.html, what it does, and how the pieces fit into the overall flow.

1. Big picture

The whole file is one static HTML+JS document of about 200 lines — no build step, no backend, just the browser talking directly to the Dependency-Track REST API. When the page loads:

- The browser downloads Tailwind from a CDN (one script tag).
- The JavaScript runs **refresh()** once automatically.
- **refresh()** makes a handful of API calls: one for the team list, one per team for its ACL-mapped projects, then two more per project (metrics and findings).
- The results get rolled into portfolio totals and rendered as 6 summary cards plus one section per team. Clicking the Refresh button re-runs that same flow.

Two hardcoded constants point at the DT instance — **API_URL** and **API_KEY** — and that is the entirety of the configuration.

2. The HTML shell

2.1 Head

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Combined Dependency-Track Report</title>
<script src="https://cdn.tailwindcss.com"></script>
<style>body{background:#0f172a;color:#e2e8f0;
    font-family:'Segoe UI',Tahoma,sans-serif}</style>
</head>
```

Standard HTML head plus two things that matter. The Tailwind Play CDN is the only external dependency: it scans the DOM at runtime, sees the utility classes the rest of the file uses (bg-slate-800, text-sky-400, etc.) and generates the matching CSS on the fly — that's why we don't ship any stylesheet of our own. The inline style block sets the dark page background, light text colour, and page font in a single line. Everything else is styled by Tailwind classes directly on the elements.

2.2 Body

```
<body class="p-5">
<div class="max-w-7xl mx-auto">

  <h1 class="text-3xl font-bold text-sky-400 text-center mb-1">
    Combined Dependency-Track Report</h1>
  <p class="text-slate-400 text-center mb-4">
    Multi-Company Vulnerability Monitoring</p>
```

```

<div class="text-center mb-6">
  <button id="refresh"
    class="bg-sky-500 hover:bg-sky-400 disabled:opacity-50
      text-slate-950 font-semibold px-6 py-2 rounded">
    [refresh icon] Refresh
  </button>
</div>

<div id="status" class="text-slate-400 text-center mb-4 min-h-[1.25rem]"></div>

<div id="summary" class="grid gap-4 mb-6 hidden"
  style="grid-template-columns:repeat(auto-fit,minmax(160px,1fr))"></div>

<div id="companies"></div>

<div id="footer" class="text-center text-slate-500 mt-8 pt-5
  border-t border-slate-700 hidden">
  Generated <span id="ts"></span>
</div>
</div>

```

The body is empty containers that the script will fill in: a centred title and subtitle, the single **#refresh** button (the only interactive element on the page), a **#status** line for messages like 'Fetching teams...' or 'Loading Acme Corp...', an empty **#summary** grid (populated later with the six totals cards), **#companies** where the script appends one section per team, and a **#footer** with a **#ts** span the script stamps with the generated timestamp. The 'hidden' class is on summary and footer so nothing flashes empty before the first fetch completes.

3. The JavaScript

3.1 Hardcoded configuration

```

const API_URL = "http://145.100.105.193:8088";
const API_KEY = "odt_lLyxR0C8_l7kaZnEobEisCo0rYSrF4uRfwVgtCspX";

```

Two constants and only two. **API_URL** must be a URL the browser can reach (not just the DT host — 'localhost' only works if you're browsing from the DT host itself). **API_KEY** is sent as X-Api-Key on every request; it needs the DT permissions VIEW_PORTFOLIO, VIEW_VULNERABILITY, and VIEW_BADGES. Both values live in the file in plain text, so anyone who can read the file gets the key.

3.2 Tiny shortcuts

```

const $ = id => document.getElementById(id);
const SEV = { CRITICAL:0, HIGH:1, MEDIUM:2, LOW:3, INFO:4, UNASSIGNED:5 };
const TOP_N = 5;

```

\$ is just a shorter `getElementById`. **SEV** maps a severity string to a numeric rank used to sort findings so the worst come first. **TOP_N** caps the number of vulnerabilities shown per project to keep cards compact.

3.3 HTML escaper

```

const esc = s => String(s ?? '').replace(/[&<>'"]/g,

```

```
c => ({'&': '&', '<': '<', '>': '>',  
      '": '"', '": '&#39;'}[c]));
```

Every value coming from Dependency-Track goes through **esc()** before being injected into the DOM. Vulnerability descriptions in particular can contain '<' or '&' — left unescaped, they would either break the page layout or, in the worst case, inject markup. **String(s ?? "")** first guards against null or undefined.

3.4 Severity and risk colour helpers

```
const sevCls = s => ({  
  CRITICAL: 'bg-red-500/20 text-red-400',  
  HIGH: 'bg-orange-500/20 text-orange-400',  
  MEDIUM: 'bg-yellow-500/20 text-yellow-400',  
  LOW: 'bg-green-500/20 text-green-400',  
  UNASSIGNED: 'bg-slate-500/20 text-slate-400'  
}[s] || 'bg-slate-500/20 text-slate-400');  
  
const riskCls = r =>  
  r >= 50 ? 'text-red-400' :  
  r >= 10 ? 'text-orange-400' : 'text-green-400';
```

sevCls(severity) returns the Tailwind class pair for that severity — a translucent background (the /20 means 20% opacity) plus a matching text colour. Unknown values fall back to the slate (grey) pair. **riskCls(score)** turns a numeric risk into a colour: red at 50+, orange from 10, green otherwise. The thresholds are arbitrary but match the dashboard's visual intent — high risk pops red on the page.

3.5 The HTTP helper

```
async function get(url) {  
  try {  
    const r = await fetch(url, { headers: { 'X-Api-Key': API_KEY } });  
    return r.ok ? r.json() : null;  
  } catch { return null; }  
}
```

A thin wrapper around fetch that adds the X-Api-Key header so every call is authenticated, returns parsed JSON on a 2xx response, and returns null on any failure — non-2xx status, network error, CORS block, JSON parse error. Returning null rather than throwing means the calling code can do **const x = await get(...) || {}** and never crash. One failed metric never breaks the whole page.

3.6 Template: the summary card

```
const sumCard = (label, value, color = 'text-sky-400') => `  
  <div class="bg-slate-800 border border-slate-700  
    rounded-xl p-5 text-center">  
    <div class="text-4xl font-bold ${color}">${value}</div>  
    <div class="text-slate-400 mt-1">${label}</div>  
  </div>`;
```

Returns the HTML string for one summary card — a large coloured number above a muted label. **color** defaults to sky-blue; the Critical and High cards override it to red and orange.

3.7 Template: the project card

```
const projectCard = p => {
```

```

const sb = p.severities;
const badges = ['CRITICAL', 'HIGH', 'MEDIUM', 'LOW'].map(s =>
  `<span class="px-2.5 py-1 rounded-full text-xs font-semibold ${sevCls(s)}">${s.slice(0,4)} ${sb[s]||0}</span>`
).join('');
const rows = (p.top || []).map(v => `
  <tr class="border-b border-slate-800">
    <td class="p-2 text-xs">${esc(v.vulnId)}</td>
    <td class="p-2 text-xs"><span class="px-2 py-0.5 rounded-full ${sevCls(v.severity)}">${esc(v.severity)}</span></td>
    <td class="p-2 text-xs">${esc(v.source)}</td>
    <td class="p-2 text-xs text-slate-300">${esc(v.description)}</td>
  </tr>`.join('')
|| `<tr><td colspan="4" class="p-3 text-center text-green-400 text-sm">No vulnerabilities</td></tr>`;
return `
<div class="bg-slate-900 border border-slate-700 rounded-lg p-4">
  <h3 class="text-sky-400 font-semibold mb-1">${esc(p.name)}
  <span class="text-slate-500 text-sm font-normal">${esc(p.version || '')}</span></h3>
  <p class="text-slate-400 text-sm mb-3">${esc(p.description)}</p>
  <div class="flex justify-between border-b border-slate-800 py-1.5">
    <span class="text-slate-400">Components</span>
    <span class="font-semibold">${p.components}</span></div>
    <div class="flex justify-between border-b border-slate-800 py-1.5">
      <span class="text-slate-400">Vulnerabilities</span>
      <span class="font-semibold">${p.vulnerabilities}</span></div>
    <div class="flex justify-between border-b border-slate-800 py-1.5">
      <span class="text-slate-400">Risk score</span>
      <span class="font-semibold ${riskCls(p.risk)}">${p.risk}</span></div>
    <div class="flex flex-wrap gap-2 my-3">${badges}</div>
    <table class="w-full border-collapse">
      <thead><tr class="border-b border-slate-700">
        <th class="text-left text-xs text-slate-400 p-2">Vuln</th>
        <th class="text-left text-xs text-slate-400 p-2">Severity</th>
        <th class="text-left text-xs text-slate-400 p-2">Source</th>
        <th class="text-left text-xs text-slate-400 p-2">Description</th>
      </tr></thead>
      <tbody>${rows}</tbody>
    </table>
  </div>`;
};

```

The biggest template, built in three parts. **badges** is the row of CRIT/HIGH/MEDI/LOW pills — for each of the four severities it generates a span styled with that severity's `sevCls()` colours, showing the first 4 letters and the count. **rows** is the body of the findings table — one row per top vulnerability with every value passed through `esc()`; if the project has no findings the row falls back to a green 'No vulnerabilities' line. The card itself is then assembled: project name + version, description, three metric rows (Components, Vulnerabilities, Risk score with `riskCls()` colour), the badge row, and the findings table. The metric rows use Flexbox (`flex justify-between`) to push the label left and the value right with a thin bottom border separating them — that's the horizontal 'list' look on the dashboard.

3.8 Template: the company section

```

const companySection = c => `
  <div class="bg-slate-800 border border-slate-700 rounded-xl p-6 mb-6">
    <div class="flex justify-between items-center mb-4 flex-wrap gap-2">
      <span class="text-2xl font-bold text-slate-100">${esc(c.name)}</span>
      <span class="bg-slate-700 px-3.5 py-1.5 rounded-full text-sm text-slate-300">
        ${c.projects.length} project${c.projects.length===1?'':'s'}</span>
    </div>
    <div class="grid gap-4" style="grid-template-columns:repeat(auto-fit,minmax(420px,1fr))">
      ${c.projects.map(projectCard).join('')}
    </div>
  </div>`;

```

Wraps that company's project cards in a panel. The header is a flex row with the team name on the left and a count badge ('2 projects') on the right. The cards themselves sit in a CSS grid that wraps to fit the window — each card wants at least 420px; if there's only room for one column, they stack.

3.9 The main flow — refresh()

This is where everything happens. The function clears the UI, fetches teams, loops through each team and project, shapes the data, computes totals, and renders. The code below is split into logical chunks with an explanation under each one.

```
async function refresh() {
  $('refresh').disabled = true;
  $('summary').classList.add('hidden');
  $('footer').classList.add('hidden');
  $('companies').innerHTML = '';
  $('status').textContent = 'Fetching teams...';
```

Sets up a clean slate: disable the button (so impatient double-clicks don't queue), hide the previous summary and footer, wipe the company sections, and put a status message up so you can tell something's happening.

```
const teams = await get(`${API_URL}/api/v1/team`);
if (!teams) {
  $('status').textContent =
    `Could not reach ${API_URL} - check the DT URL, the API key, and that CORS is enabled.`;
  $('refresh').disabled = false; return;
}
```

The first API call. If `get()` returned null, the most likely culprit is one of three things — wrong URL, wrong key, or CORS not enabled on the apiserver — and the status line says so explicitly. The button is re-enabled so you can fix the problem and retry.

```
const companies = [];
for (const t of teams) {
  $('status').textContent = `Loading ${t.name}...`;
  const projects = await get(
    `${API_URL}/api/v1/acl/team/${t.uuid}?pageSize=500&pageNumber=1`) || [];
  if (!projects.length) continue;
```

For each team, fetch the projects assigned to it via ACL. `pageSize=500` is a one-shot upper bound — we don't paginate beyond 500 projects per team. If a team has no projects (for example the default Administrators team), `continue` skips it. That's why empty teams never appear as empty sections on the page.

```
const projList = [];
for (const p of projects) {
  const m = await get(
    `${API_URL}/api/v1/metrics/project/${p.uuid}/current`) || {};
  const findings = await get(
    `${API_URL}/api/v1/finding/project/${p.uuid}`) || [];
  findings.sort((a, b) =>
    (SEV[(a.vulnerability || {}).severity || 'UNASSIGNED'] ?? 9) -
    (SEV[(b.vulnerability || {}).severity || 'UNASSIGNED'] ?? 9));
```

For each project, two calls in series — the current metrics snapshot (component count, vulnerabilities by severity, inheritedRiskScore) and the individual findings matched to this project's components. The sort uses the SEV rank: criticals first, then high, etc. The **?? 9** handles unknown severities by sending them to the end.

```
projList.push({
  name: p.name, version: p.version, description: p.description || '',
  components: m.components || 0,
  vulnerabilities: m.vulnerabilities || 0,
  risk: m.inheritedRiskScore || 0,
  severities: {
    CRITICAL: m.critical || 0, HIGH: m.high || 0,
    MEDIUM: m.medium || 0, LOW: m.low || 0,
    UNASSIGNED: m.unassigned || 0
  },
  top: findings.slice(0, TOP_N).map(f => {
    const v = f.vulnerability || {};
    return { vulnId: v.vulnId, severity: v.severity, source: v.source,
      description: (v.description || '').slice(0, 220) };
  })
});
}
companies.push({ name: t.name, projects: projList });
}
```

Two pieces of important data shaping. A flat project record is built with exactly the fields `projectCard()` consumes — this decouples the template from DT's raw response shape. **`findings.slice(0, TOP_N)`** keeps only the top 5, and each finding's description is truncated to 220 characters so it fits cleanly in the table cell. The team gets pushed as `{name, projects}`.

```
if (!companies.length) {
  $('status').textContent = 'No teams with projects found.';
  $('refresh').disabled = false; return;
}
```

Fallback message if every team was skipped. Means DT is reachable but nothing is configured.

```
const tot = companies.reduce((a, c) => {
  a.projects += c.projects.length;
  for (const p of c.projects) {
    a.comps += p.components; a.vulns += p.vulnerabilities;
    a.crit += p.severities.CRITICAL; a.high += p.severities.HIGH;
  }
  return a;
}, { projects: 0, comps: 0, vulns: 0, crit: 0, high: 0 });
```

One pass over all companies and projects to compute the six portfolio totals that go into the summary cards.

```
$('summary').innerHTML = [
  sumCard('Companies',      companies.length),
  sumCard('Projects',       tot.projects),
  sumCard('Components',     tot.comps),
  sumCard('Vulnerabilities', tot.vulns),
  sumCard('Critical',       tot.crit, 'text-red-500'),
  sumCard('High',          tot.high, 'text-orange-500')
].join('');
```

```

$('summary').classList.remove('hidden');
$('#companies').innerHTML = companies.map(companySection).join('');
$('#ts').textContent = new Date().toLocaleString();
$('#footer').classList.remove('hidden');
$('#status').textContent = '';
$('#refresh').disabled = false;
}

```

The render step. Build the six summary cards as one HTML string and drop it into `#summary`, then un-hide it. Map each company through `companySection()` and drop the joined HTML into `#companies`. Stamp the footer with the current local time and un-hide it. Clear the status line and re-enable the Refresh button.

3.10 Wire-up

```

$('#refresh').addEventListener('click', refresh);
refresh(); // auto-fetch on first load

```

Two lines, big impact. The button is wired to `refresh()` — that's the Refresh behaviour. The bare `refresh()` call at the end means the page fetches on load; you don't have to click anything to see the first dashboard.

4. How a single click maps to what you see

For three teams with two projects each, one click runs **16 API calls**:

```

GET /api/v1/team                                1
GET /api/v1/acl/team/{acme-uuid}?pageSize=500&pageNumber=1  1
GET /api/v1/acl/team/{beta-uuid}?pageSize=500&pageNumber=1  1
GET /api/v1/acl/team/{gamma-uuid}?pageSize=500&pageNumber=1 1
GET /api/v1/metrics/project/{uuid}/current    (x6 projects) 6
GET /api/v1/finding/project/{uuid}            (x6 projects) 6
---
16

```

All sequential. On a fast network that's about 3-5 seconds total; on a slow one, longer — which is why the status line is updated at each major step. The returned data lands in this rough shape before rendering:

```

[
  { name: "Acme Corp", projects: [
    { name, version, description,
      components, vulnerabilities, risk,
      severities: { CRITICAL, HIGH, MEDIUM, LOW, UNASSIGNED },
      top: [ { vulnId, severity, source, description }, ... up to 5 ] },
    ...
  ]},
  ...
]

```

The render then loops over that array, calling `companySection` for each entry, which in turn calls `projectCard` for each project, which assembles the badges and table rows. All of it is plain string concatenation — there's no virtual DOM, no framework, just `innerHTML` and template literals.

5. What can go wrong, and what you'll see

Symptom on the page	Most likely cause
'Could not reach <url> — check the DT URL, the API key, and that CORS is enabled'	DT URL is wrong, the API key is wrong or expired, or the DT host doesn't allow CORS.
'No teams with projects found.'	The page reached DT successfully, but every team has zero ACL-mapped projects. Add projects.
Page renders but every card shows 0 vulnerabilities	DT is running and analysed the SBOMs, but no vulnerabilities matched. Almost always means 0.
Components / risk all zero on one project.	DT hasn't finished analysing yet. Click Refresh again in a minute, or re-upload the BOM.
Some teams missing entirely.	Those teams have no projects in their ACL (intentional — keeps the page tidy).

6. Why it's structured this way

- **One file, no build step.** Tailwind via CDN means no npm, no webpack, no package.json. Anyone can open report.html, change the two constants, and use it.
- **Plain fetch and innerHTML.** No framework — easier to read, audit and modify. The trade-off (manual escaping) is handled by esc().
- **Templates return strings, then we use innerHTML.** This is fine because every user-controlled value goes through esc(). It's also dramatically simpler than building DOM nodes one at a time.
- **Errors are silent (return null).** A single broken project should not break the whole page; the rest of the dashboard keeps rendering with what it has.
- **The button is the refresh.** No polling, no auto-refresh interval. You decide when fresh data matters — which avoids hammering the DT API.

That's the whole file: a static page, two constants, a handful of templates, and one orchestrating function that turns Dependency-Track's API responses into the dashboard you look at.